PYTHON PLAYWRIGHT CHEATSHEET

1. INSTALLATION

```
# Install Playwright
pip install playwright

# Install browsers
playwright install
```

2. BASIC SETUP

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("https://example.com")
    browser.close()
```

3. LAUNCH BROWSER

```
browser = p.chromium.launch(
    headless=True,          # Headless mode
    slow_mo=50,             # Slow down (ms)
    args=["--start-maximized"],
    channel="chrome"        # Use Chrome
)
```

4. BROWSER CONTEXT

```
context = browser.new_context(
    viewport={"width": 1280, "height": 720},
    user_agent="Mozilla/5.0 ...",
    locale="en-US",
    timezone_id="America/New_York",
    ignore_https_errors=True,
    storage_state="state.json"  # For reuse
)

# Save storage state
context.storage_state(path="state.json")
```

5. NAVIGATION

```
page.goto("https://example.com")     # Open URL
page.reload()                        # Reload pa
ge
page.go_back()                       # Back
page.go_forward()                    # Forward
page.wait_for_url("**/dashboard")    # Wait for
URL
page.wait_for_load_state("networkidle") # Wait f
or network idle
```

6. LOCATORS (AUTO-WAIT)

```
page.locator("text=Submit")                 # By
 text
page.locator("#id")                          # B
y ID
page.locator(".class")                       # B
y class
page.locator("[data-testid='btn']")         # By
 data attribute
page.get_by_role("button", name="Save")      # B
y ARIA role
page.get_by_text("Hello")                    # B
y text
page.get_by_label("Email")                   # B
y label
page.get_by_placeholder("Search...")         # B
y placeholder
page.locator("xpath=//div[@class='item']")   # B
y XPath
```

7. ACTIONS

```
locator.click()                                # C
lick
locator.dblclick()                             # D
ouble click
locator.fill("text")                          # Fi
ll input
locator.type("Hello", delay=100)               # T
ype with delay
locator.clear()                                # C
lear input
locator.check()                                # C
heck checkbox/radio
locator.uncheck()                              # U
ncheck
locator.select_option(value="1")              # Se
lect by value
locator.hover()                                # H
over
locator.press("Enter")                         # P
ress key
locator.set_input_files("path/to/file.txt")  # U
pload file
```

8. ASSERTIONS

```
from playwright.sync_api import expect

expect(locator).to_be_visible()
expect(locator).to_have_text("Hello")
expect(locator).to_contain_text("Hello")
expect(locator).to_have_value("123")
expect(locator).to_be_enabled()
expect(locator).to_be_disabled()
expect(locator).to_be_checked()
expect(locator).to_have_count(3)
expect(page).to_have_url("**/dashboard")
expect(page).to_have_title("Dashboard")
```

9. DYNAMIC CONTENT

```
page.wait_for_selector(".item", state="visible")
page.wait_for_timeout(1000)                    # (
Avoid using)
page.wait_for_response(lambda r: "api/data" in r
.url and r.status == 200)
page.wait_for_request("**/event")
page.wait_for_function("() => document.querySele
ctor('.done')")
```

10. WORKING WITH FRAMES

```
frame = page.frame(name="frame-name")
frame = page.frame(url="**/frame.html")
frame_locator = page.frame_locator("#iframeId")
frame_locator.locator("text=Click").click()
page.frames                                       # L
ist all frames
```

11. WINDOWS & POPUPS

```
with page.expect_popup() as popup_info:
    page.click("text=Open Window")
popup = popup_info.value
popup.wait_for_load_state()
popup.close()
```

12. FILE DOWNLOADS

```
with page.expect_download() as download_info:
    page.click("#download")
download = download_info.value
download.save_as("path/to/download/file.pdf")
```

13. SCREENSHOTS & PDF

```
page.screenshot(path="screenshot.png", full_page
=True)
locator.screenshot(path="element.png")
page.pdf(path="page.pdf", format="A4", print_bac
kground=True)
```

14. TRACING & DEBUGGING

```
context.tracing.start(screenshots=True, snapshot
s=True, sources=True)
# ... actions ...
context.tracing.stop(path="trace.zip")

# PWDEBUG=1 python script.py       # Debug mode
# page.pause()                     # Inspector
# playwright show-trace trace.zip
```

15. ENV & CONFIG

```
# Headless: True (CI) / False (Local)
# context = browser.new_context(base_url="https:
//app.com")
# ignore_https_errors=True
# context = browser.new_context(proxy={"server":
 "http://host:port"})
# context = browser.new_context(record_video_dir
="videos/")
```

■ **PRO TIPS:**

- Locators are strict and resilient—prefer them over raw selectors.
- Auto-wait is on by default—use assertions & waits only when explicitly required.
- Always favor built-in network waits over static, hardcoded timeouts.
- Trace everything in your Continuous Integration (CI) environment for simplified debugging.